

CSCE 608 DATABASE SYSTEMS · MODULE 03 · LECTURE NOTES

B+Trees

The data structure the storage hierarchy forced into existence.

MODULE OUTCOMES

- Explain why a B+tree rather than a binary tree.
- Implement insert with splits and delete with merges.
- Compute tree height from fanout and explain what it costs.
- Distinguish clustered from non-clustered indexes.
- Explain when an index helps and when it does not.

1 Why not a binary tree

THE COST MODEL IS I/Os, NOT COMPARISONS

A balanced binary search tree over a million keys has height about 20. In an in-memory setting that is excellent — twenty comparisons.

On disk, each level is a pointer chase to an arbitrary address, which is a random read. Twenty random reads at 100 μ s each is 2 ms for a single lookup, and the comparisons themselves are free by comparison.

The cost model that matters is the number of I/Os.

A page holds hundreds or thousands of keys, and reading a page costs the same as reading a byte. So make each node a page, and let it have as many children as fit.

With fanout 400, a million keys fit in three levels. Three I/Os rather than twenty — and because the root and the level below it are only a few megabytes, they stay resident in the buffer pool permanently, so a lookup is effectively *one* I/O. **The shape of the structure is forced by the hardware**, which is why every disk-based index is some variation on it.

$$\text{height} = \lceil \log_f(N) \rceil \quad f \approx \text{PAGE_SIZE} / (\text{key_size} + \text{pointer_size})$$

Page size	Entry size	Fanout f	Keys at height 3
4 KB	16 bytes	~250	$\sim 1.5 \times 10^7$
16 KB	16 bytes	~1000	10^9
16 KB	64 bytes (string keys)	~250	$\sim 1.5 \times 10^7$

A billion keys in three levels, with the top two levels cached. This single calculation explains why B+trees displaced everything else for disk-resident indexing, and it is worth doing for your own page size and key type rather than taking it on trust.

	B-tree	B+tree
Values stored	In every node, internal and leaf.	Only in leaves. Internal nodes hold keys and child pointers only.
Lookup	May terminate early at an internal node.	Always descends to a leaf. Predictable cost , always the height.
Fanout	Lower — values consume space in internal nodes.	Higher , so the tree is shallower for the same key count.
Range scan	Awkward — requires in-order traversal back up and down the tree.	Leaves are linked in key order , so a range scan finds the start and then walks sequentially. This is the decisive advantage , because range scans are a large fraction of what databases actually do.

2 Operations

```

INSERT(key, value):
  leaf = descend from root, choosing child by key range
  if leaf has room: insert in sorted position; done
  else:
    allocate new leaf
    move upper half of entries into it
    link it into the leaf chain
    COPY the separator key up into the parent
    if parent now full, split it too (recursively)
    if the ROOT splits, allocate a new root <-- the only way
                                              the tree grows taller

```

THE TREE GROWS AT THE ROOT, WHICH IS WHY IT STAYS BALANCED

A B+tree never rebalances. There is no rotation, no colour, no rebalancing pass.

It stays perfectly balanced because it grows at the root rather than at the leaves. Every leaf is at the same depth by construction: when a split propagates all the way up and the root itself splits, a new root is created above it and every path lengthens by one simultaneously.

Copy versus move is the detail people get wrong. In a B+tree, the separator key pushed into the parent is **copied**, not moved — the key must remain in the leaf, because that is where its value lives. In a B-tree it is moved, because the value moves with it. Getting this backwards produces a tree that loses keys, and it is the classic implementation bug.

DELETION IS WHERE REAL IMPLEMENTATIONS CHEAT

The textbook algorithm merges a node with a sibling when its occupancy falls below half, or redistributes entries between them if the sibling has enough to spare, propagating merges upward as needed.

Most production implementations do not do this. They remove the entry and leave the node underfull.

The reasons are practical: merging is considerably more complex than splitting; it requires locking more nodes, which complicates concurrency (Module 04); and in most workloads deletions are followed by insertions that refill the space, so the work would be wasted.

The cost is gradual degradation. A tree subjected to many deletions becomes sparse — more pages than the data requires, so more I/O per scan and sometimes a taller tree. **This is why REINDEX and VACUUM FULL exist**, and why they periodically need to be run on write-heavy tables.

- **Prefix compression.** Keys within a node frequently share a prefix — URLs, names, timestamps. Storing the common prefix once per node and only the differing suffixes per entry can **double the fanout** on string keys, which removes a level from the tree.
- **Suffix truncation.** A separator key in an internal node only has to *distinguish* the two subtrees, not match any actual key. It can be truncated to the shortest prefix that separates them, which saves space in exactly the nodes where space buys the most.
- **Bulk loading.** Building a tree by repeated insertion is slow and produces a tree about 70% full. Sorting the data first and building bottom-up is far faster and produces a densely packed tree — which is what CREATE INDEX does.
- **Variable-length keys** make 'half full' ambiguous — half by entry count or half by bytes? — and complicate split-point selection.
- **Duplicate keys** are handled either by storing a list of record IDs per key, or by appending the record ID to the key to make every key unique. The second is simpler and is what most systems do.

3 Using indexes

	Clustered index	Non-clustered index
Structure	The table is the index. Rows are stored in the leaves, in key order.	Leaves hold keys and pointers to rows stored elsewhere.
Range scan	Sequential — the rows are physically adjacent. Excellent.	One random read per row to fetch it from the table. Can be slower than a full scan.
How many	One per table , necessarily — rows can only be in one physical order.	As many as you like.
Insert cost	Insertions in random key order fragment the table itself.	The table is unaffected; only the index fragments.
Who uses it	MySQL InnoDB (always, on the primary key); SQL Server by default.	PostgreSQL's only kind — it has no clustered index at all, which is a real and frequently underestimated difference.

A COVERING INDEX AVOIDS THE TABLE ENTIRELY

If an index contains *every* column a query references — both in the predicate and in the select list — then the query can be answered from the index alone and **the base table is never read**.

This is an index-only scan, and it is frequently an order of magnitude faster than an index scan that must fetch each matching row, because it eliminates one random read per result row.

The cost is a wider index: more disk space, more work on every write, and lower fanout, which can add a level to the tree.

INCLUDE columns are the refinement. They let you add payload columns to an index without making them part of the sort key — you get coverage without paying for a wider key in every internal node. PostgreSQL, SQL Server, and others support them, and they are underused.

An index does not help when	Why
The query returns a large fraction of the table.	Each matching row costs a random read; past some selectivity a sequential scan of everything is cheaper. The crossover is often as low as 5–10% on spinning disks , and higher on SSDs where random reads are less punishing.
The column has low cardinality.	An index on a boolean or a status column with four values points at huge row sets, so every lookup degenerates into the case above.
The leading column is not in the predicate.	See below. The most common and most expensive indexing mistake.
A function is applied to the column.	WHERE upper(name) = 'SMITH' cannot use an index on name, because the index is ordered by name and not by upper(name). An expression index on upper(name) fixes it , and so does not writing the query that way.

An index does not help when	Why
The workload is write-heavy.	Every index must be updated on every insert, update, and delete that touches its columns. Indexes are not free; they move cost from reads to writes.

COMPOSITE INDEX ORDER IS NOT SYMMETRIC

An index on (a, b) is sorted by a, and then by b *within* each distinct value of a. It is a single ordering, not two independent ones.

It serves WHERE a = ?, and WHERE a = ? AND b = ?, and WHERE a = ? AND b BETWEEN ? AND ? — anything that fixes a prefix of the key.

It does not serve WHERE b = ?. The rows with a given b value are scattered across the index in as many separate places as there are distinct values of a. The optimiser may still scan the whole index if it is narrower than the table, but that is a scan, not a seek.

So (a,b) and (b,a) are genuinely different indexes, and which you want depends on the queries you run.

The usual rule: equality predicates first, then the range predicate, then any covering columns. A range predicate on a column stops the index from being useful for anything after it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lectures 7–8, Tree Indexes	B+tree structure, splits, merges, and the design decisions in §2. The project assignment is the implementation.
Graefe — Modern B-Tree Techniques (free)	The definitive survey. Prefix compression, suffix truncation, bulk loading, and everything else in §2's refinements list.
Use The Index, Luke (free online book)	The best free material on §3 — when indexes help, composite ordering, and covering indexes, with examples in several dialects.
Database Internals — B-tree chapters	A second treatment with more attention to on-disk layout details.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Compute the fanout and height for your page size and key type, for 10^6 , 10^9 , and 10^{12} keys. Compare against a binary tree.

- 2 Implement a B+tree with insert and point lookup. Verify against an in-memory map after every operation on a randomised workload.
- 3 Implement splits correctly and demonstrate that the tree grows only at the root — assert that all leaves are at equal depth after every insert.
- 4 Deliberately move the separator key up instead of copying it, and demonstrate the keys that are lost.
- 5 Implement range scan over linked leaves. Measure pages read and compare against the analytic prediction.
- 6 Implement delete without merging. Insert a million keys, delete 90% in random order, and report the resulting page count against the theoretical minimum.
- 7 Implement bulk loading. Compare build time and resulting page count against repeated insertion of the same data.
- 8 Implement prefix compression and report the fanout improvement on a column of URLs or file paths.
- 9 In PostgreSQL, create a table with a composite index on (a,b). Use EXPLAIN to confirm which of four predicates use the index.
- 10 Find the selectivity crossover on your own machine: at what fraction of rows returned does a sequential scan beat an index scan?

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is a binary tree unsuitable for a disk-based index?
- 2 Write the height formula and compute it for a billion keys at fanout 1000.
- 3 Give four differences between a B-tree and a B+tree, and the decisive one.
- 4 Why does a B+tree stay balanced without any rebalancing logic?
- 5 Why is the separator key copied rather than moved?
- 6 Why do most implementations skip merging on delete, and what is the cost?
- 7 Compare clustered and non-clustered indexes on five axes.
- 8 What is a covering index, what does it save, and what does INCLUDE add?
- 9 Give five situations where an index does not help.
- 10 Why does an index on (a,b) not serve a predicate on b alone?